

System Scanner Pro v3.0

Admin-Client Connection Guide

Step-by-Step Setup & Configuration

AI-Powered Distributed Endpoint Monitoring

Document Version:	2.0
Application Version:	System Scanner Pro v3.0
Technology Stack:	Django 5.x + DRF + Django Channels + Python 3.10+
Communication:	HTTP REST + WebSocket + Cloud Discovery (Supabase) + UDP

Table of Contents

1. Architecture Overview
2. Cloud Discovery Setup (Supabase)
3. Prerequisites & Requirements
4. Install Dependencies
5. Configure Environment
6. Start the Admin Server
7. Start the Client Agent
8. Approve the Client
9. Verify Connection
10. Connection Methods Reference
11. Troubleshooting
12. Port Reference

1. Architecture Overview

System Scanner Pro uses a **distributed client-server architecture**. The admin server (Django) runs on a central machine and serves a web dashboard, REST API, and WebSocket endpoint. Client agents run on each monitored machine, connecting to the admin server via HTTP and WebSocket.

Component	Technology	Purpose
Admin Server	Django 5.x + DRF	Web dashboard, REST API, data storage
WebSocket Layer	Django Channels	Real-time bidirectional communication
Client Agent	Python (stdlib + websockets)	Hardware scanning, event monitoring
Cloud Discovery	Supabase REST API	Admin IP registry for cross-network client discovery
Auto-Discovery	UDP broadcast (port 45000)	LAN-only fallback for same-network discovery
Database	SQLite (default) / PostgreSQL	Client data, scan results, alerts

Communication Flow:

- Admin detects public IP and registers it in Supabase cloud registry on startup
- Admin re-registers IP every 5 minutes to handle IP changes
- Client queries Supabase to find admin's current IP (works across any network)
- Fallback: client tries cached URL, then UDP broadcast (LAN), then manual prompt
- Client registers via HTTP POST to admin REST API
- Admin approves the client from the dashboard
- Client starts heartbeat (HTTP) + WebSocket connection
- Real-time events flow: USB, File, Process, Software changes

2. Cloud Discovery Setup (Supabase)

Cloud discovery allows clients to find the admin server **across any network**, even when the admin's IP address changes. The admin registers its public IP in a Supabase table, and clients query that table on startup and periodically.

How it works:

- 1. Admin detects its public IP via external service (ipify.org)
- 2. Admin writes IP to Supabase table `server_registry`
- 3. Admin re-registers every 5 minutes (handles IP changes)
- 4. Client queries Supabase on startup to find admin URL
- 5. Client re-checks Supabase every 5 minutes for IP changes
- 6. If Supabase is unavailable, falls back to cached URL then UDP

Step 2.1: Create Supabase Table

Go to your Supabase project dashboard → SQL Editor and run the SQL from `setup_cloud_discovery.sql`:

```
CREATE TABLE server_registry (  
  id TEXT PRIMARY KEY DEFAULT 'admin',  
  ip_address TEXT NOT NULL,  
  port INTEGER DEFAULT 80,  
  protocol TEXT DEFAULT 'http',  
  is_active BOOLEAN DEFAULT true,  
  updated_at TIMESTAMPTZ DEFAULT now()  
);  
  
-- Enable RLS with public read, service-role write  
ALTER TABLE server_registry ENABLE ROW LEVEL SECURITY;
```

Step 2.2: Configure Supabase Credentials

Ensure the `.env` file has your Supabase credentials:

```
SUPABASE_URL="https://your-project.supabase.co"  
SUPABASE_SERVICE_KEY="your-service-role-key"
```

Step 2.3: Admin Auto-Registers on Startup

When the admin server starts, it automatically detects its public IP and registers it in Supabase. No manual configuration needed. The registration refreshes every 5 minutes to handle IP changes.

Step 2.4: Client Auto-Discovers on Startup

When the client starts, it queries Supabase to find the admin's current IP. If found, it uses that URL. If not (Supabase down), it falls back to cached URL, then UDP broadcast, then manual prompt. The client also re-checks Supabase every 5 minutes for IP changes.

Discovery Fallback Chain:

- 1. Cloud discovery (Supabase) - works across any network
- 2. Cached URL (client_config.json) - from previous successful connection
- 3. UDP broadcast (port 45000) - LAN only
- 4. Manual URL prompt - last resort

3. Prerequisites & Requirements

Requirement	Details
Python	Version 3.10 or higher
pip	Python package manager (included with Python 3.4+)
Operating System	Windows 7/10/11, Linux, or macOS
Network	Admin and client machines must be on the same network (or have routing between them)
Ports	TCP 80 (or custom port) for admin, UDP 45000 for auto-discovery
Admin Privileges	Port 80 requires elevated privileges; use port 8080 as alternative
Disk Space	~50 MB for application + scan data

4. Install Dependencies

Step 3.1: Open a Terminal / Command Prompt

Navigate to the project directory:

```
cd "C:\new intern project\system_scanner_pro\admin-client"
```

Step 3.2: Install Python Packages

Install all required packages from requirements.txt:

```
pip install -r requirements.txt
```

Key packages installed:

- django, djangorestframework, channels, daphne (admin server)
- apscheduler (scheduled scanning)
- PyJWT (authentication tokens)
- websockets, watchdog (client agent)
- reportlab (PDF report generation)

Note: If you get permission errors, try: `pip install --user -r requirements.txt`

5. Configure Environment

Step 5.1: Environment File (.env)

The `.env` file in the project root contains Django settings. The defaults work for local development:

```
DJANGO_SECRET_KEY="django-insecure-change-me-in-production-abc123"
DJANGO_DEBUG=True
DJANGO_ALLOWED_HOSTS="*"

# Supabase (Cloud Discovery)
SUPABASE_URL="https://your-project.supabase.co"
SUPABASE_SERVICE_KEY="your-service-role-key"
```

Step 5.2: Admin Server Configuration

The admin server reads `admin/admin_config.json` for bind address. Default is `0.0.0.0` (all interfaces).

Step 5.3: Client Configuration

The client reads `client_config.json`. The `admin_url` field is set automatically via cloud discovery.

Field	Default	Description
<code>admin_url</code>	"" (empty)	URL of the admin server (set on first run)
<code>scan_interval</code>	3600	Seconds between automatic scans (1 hour)
<code>auto_start</code>	true	Start scanning automatically on client boot

6. Start the Admin Server

Step 6.1: Run the Admin Server

Execute the following command from the project root:

```
python admin/main.py
```

Alternative options:

- Custom port: `python admin/main.py --port 8080`
- Specific host: `python admin/main.py --host 192.168.1.100`
- Reset IP config: `python admin/main.py --reset`
- Change default credentials: `python admin/main.py --username myadmin --password mypass`

Step 6.2: First-Run Setup Prompts

On first run, the server will:

- **Database migration** - Creates SQLite database at `admin/data/scanner.db`
- **Admin user creation** - Creates default superuser: `admin / admin123`
- **IP address prompt** - Enter `0.0.0.0` (all interfaces) or a specific IP
- **Cloud discovery registration** - Detects public IP and registers in Supabase
- **UDP discovery start** - Begins broadcasting on port `45000`
- **Django server start** - Serves on the configured `host:port`

Step 6.3: Expected Console Output

You should see output similar to:

```
=====
System Scanner Pro Admin Panel v2.1
(Django + DRF + Bootstrap 5)
=====

Using saved IP: 0.0.0.0
Running database migrations...
Admin user created: admin / admin123
Admin client key:

Dashboard: http://0.0.0.0:80
```

```
Login: http://0.0.0.0:80/login/
```

```
UDP discovery active on port 45000 (listen + broadcast)
```

```
[OK] Registered with cloud discovery: http://1.2.3.4:80
```

```
Cloud discovery refresh every 300s
```

Step 6.4: Open the Dashboard

Open your web browser and navigate to **http://localhost** (or `http://<server-ip>`). Log in with **admin / admin123**.

7. Start the Client Agent

The client agent can run on the **same machine** as the admin server (for testing) or on **any other machine** on the same network.

Step 7.1: Run the Client Agent

Open a new terminal and navigate to the project root, then:

```
python client/main.py
```

Step 7.2: First-Run Configuration

On first run, the client will:

- **Generate registration key** - Unique key saved to client_key.json
- **Generate hardware fingerprint** - Device identity saved to client_key.json
- **Cloud discovery** - Query Supabase for admin's current IP address
- **Fallback chain** - Try cached URL, UDP broadcast, then manual prompt

Admin URL resolution (automatic):

- 1. Cloud discovery via Supabase (works across any network)
- 2. Cached URL from previous session (client_config.json)
- 3. UDP broadcast detection (LAN only)
- 4. Manual URL prompt (last resort)

Step 7.3: Registration Process

The client will automatically:

1. Send registration request to admin server
2. Wait for admin approval (polls every 5 seconds)
3. Once approved, perform initial hardware scan
4. Submit scan results to admin server
5. Start heartbeat loop (every 30 seconds)
6. Connect WebSocket for real-time commands
7. Start event monitors (USB, File, Process, Software)

Step 7.4: Expected Console Output

You should see output similar to:

```
=====
System Scanner Pro Client v1.0.0
=====
```

Your Registration Key:

Device Fingerprint:

Admin Server: http://localhost:80

Connecting to admin server...

[WAITING] Registration sent. Waiting for admin approval...

[OK] Admin approved registration.

Performing initial scan...

[OK] Initial scan submitted successfully!

Starting communication channels...

[OK] Heartbeat watchdog started

Connecting WebSocket for real-time communication...

WebSocket client started (auto-reconnect enabled)

[OK] 4 event monitors active

8. Approve the Client

Step 8.1: Open the Admin Dashboard

In your web browser, go to **http://localhost** and log in with **admin / admin123**.

Step 8.2: Navigate to Clients Page

Click on **Clients** or **Dashboard** in the navigation menu. You will see the pending client listed with a "Pending" status badge.

Step 8.3: Approve the Client

Click on the client entry and click the **Approve** button, or use the bulk action checkbox to approve multiple clients at once. Once approved, the client will automatically detect the status change and proceed.

Tip: The client polls the admin server every 5 seconds to check its approval status. After approval, it immediately performs an initial scan and begins normal operation.

9. Verify Connection

After approval, verify that everything is working correctly:

- **Dashboard** - The client should appear in the admin dashboard with a green "Online" status badge.
- **Scan Data** - Click on the client to view its hardware scan results (CPU, RAM, GPU, storage, OS).
- **Heartbeat** - The "Last Seen" timestamp should update every ~30 seconds.
- **WebSocket** - Real-time updates should appear without page refresh (device changes, alerts).
- **Event Monitors** - USB insertion/removal, process changes, and software changes should appear in the monitoring page.
- **Scheduled Scan** - The client performs scans at the configured interval (default: 1 hour).

***Note:** The first scan takes a few seconds. Subsequent scans are faster. You can trigger an on-demand scan from the admin dashboard using the "Scan Now" button.*

10. Connection Methods Reference

Method	Protocol	Direction	Purpose
Cloud Discovery	HTTPS	Client → Supabase	Find admin IP across any network (primary method)
UDP Auto-Discovery	UDP	Bidirectional	LAN-only fallback for same-network discovery
Registration	HTTP POST	Client → Admin	Client registers with admin server
Status Check	HTTP GET	Client → Admin	Client polls for approval status
Heartbeat	HTTP POST	Client → Admin	Periodic keep-alive (every 30s)
Scan Submission	HTTP POST	Client → Admin	Upload hardware scan results
WebSocket	WS/WSS	Bidirectional	Real-time commands, events, alerts
Dashboard	HTTP	Browser → Admin	Web UI for monitoring and management
REST API	HTTP	Any → Admin	Programmatic access with JWT/API key auth

Resilience Features:

- Cloud discovery - admin IP auto-registered in Supabase, refreshed every 5 minutes
- Client re-checks Supabase every 5 minutes for admin IP changes
- Exponential backoff on HTTP failures (5s → 10s → 20s → 30s max)
- Offline event queue - events saved to disk when disconnected, replayed on reconnect
- WebSocket auto-reconnect with exponential backoff
- Heartbeat watchdog thread restarts heartbeat if it crashes
- Cloud → cached → UDP → manual fallback chain for discovery

11. Troubleshooting

Problem	Cause	Solution
Cloud discovery fails	Supabase table missing or credentials wrong	Run setup_cloud_discovery.sql in Supabase SQL Editor; check .env credentials
Admin IP not updating	Public IP detection failed	Check internet connectivity; admin logs show detection errors
Port 80 permission error	Port 80 requires admin/root privileges	Use --port 8080 or run as administrator
Client cannot reach admin	Firewall or network isolation	Ensure ports 80 and 45000 are open, or use --port with a high port
Client stuck on "Waiting for approval"	Admin has not approved the client	Open admin dashboard and approve the pending client
No scan data appearing	Scan interval not elapsed or scan failed	Click "Scan Now" on the dashboard to trigger manual scan
WebSocket not connecting	Monitoring agent registration failed	Check admin server logs; restart both admin and client
UDP auto-discovery not working	Multicast/broadcast blocked by network	Enter admin URL manually on client first run
Database errors	Corrupted or missing SQLite database	Delete admin/data/scanner.db and restart admin server

12. Port Reference

Port	Protocol	Service	Direction
80	TCP	Django web server (dashboard + API + WebSocket)	Inbound (from clients and browsers)
45000	UDP	Auto-discovery broadcast/listener	Bidirectional (broadcast + receive)
8080	TCP	Alternative admin port (if configured)	Inbound (from clients and browsers)

This guide was generated for **System Scanner Pro v3.0**. For additional documentation, see the README.md file in the project root.